

Date: 27/11/2024 2 PM

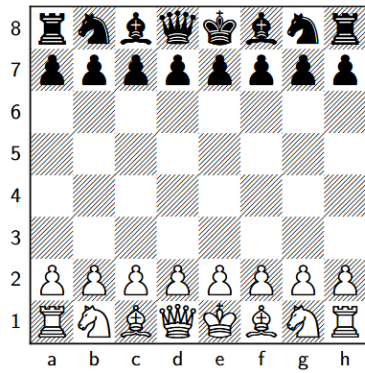
Duration: 2 hour 30 mins

Total marks: 40

Instructions

- All programs should be compatible with GNU C compiler.
- There will be a plagiarism check. Copying from internet will also be considered as plagiarism.
- Late submissions will attract marks deduction.
- You are advised to compile the code before submitting them to WeLearn. If a code does not compile at our end, marks will be deducted for the problem.
- Each program should follow a strict naming convention: **QNo.c** (e.g. Q1.c, Q2a.c etc.). Programs not adhering to the convention will not be considered for evaluation.
- All codes (.c files) should be submitted in a single zipped folder (folder name containing your WeLearn ID e.g. 17MS001) to WeLearn.
- You should put appropriate comments in the code. Importantly, junctures in the code that involve critical decision-making must be accompanied with comments to enhance the readability of the same.

1. **(Marks: 8)** Write a C program to consider an unsorted array of numbers (no need to take user input), and print the second smallest number in the array on the console. You CAN NOT sort the array.
2. **(Marks: 10)** Assume that `arr` is an unsorted array of n distinct positive integers with maximum value k ($k > 0$ is an integer). Implement an algorithm $\mathcal{A}$ of sorting `arr` with time complexity $\mathcal{O}(n + k)$ in C programming language.
[Hint: You should need another array of size k .]
3. **(Marks: 8)** Write a C program to check if an array `Array` is *contiguously completely contained* in another array `Array`. E.g. `Array` as $\{5, 2\}$ is *contiguously completely contained* in `Array` as $\{5, 2, 6, 8, 10\}$; however, `Array` as $\{5, 8\}$ or $\{5, 19\}$ are **NOT** *contiguously completely contained* in `Array` as $\{5, 2, 6, 8, 10\}$. The arrays in question are not sorted and you are NOT allowed to sort them.
4. **(Marks: 14)** Consider the game of Chess where Figure 1 shows the notations. From the starting position, consider three opening moves in Figure 2. Suppose two players are playing online chess in a website `Chaturanga.com` in a friendly mode where any number of valid UNDO operations are allowed. As an example for Figure 2, from *move 3* arrangement, by two UNDO operations, one can reach *move 1*. Implement the UNDO functionality using `STACK` data structure in C programming language. That is, each element (to be PUSHed or POPed) in the data structure is a chessboard arrangement of pieces. Note that you can start with any chess board arrangement (valid or invalid) with an arbitrarily smaller number of pieces. You may ask each player, in turn, to input an arrangement for each move and continue this for the desired number of moves. At each move, you may ask a player to continue playing (if yes, you ask for the arrangement), UNDO (if yes, ask for the number of such operations), or quit the game.



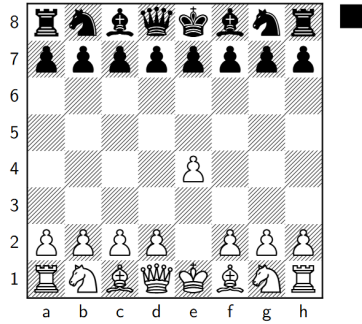
(a) Start position

Symbol	Name	Value
= K	King	Infinite
= Q	Queen	9 pawns
= R	Rook	5 pawns
= B	Bishop	3 pawns
= N	Knight	3 pawns
= P	Pawn	1 pawn

(b) Notations

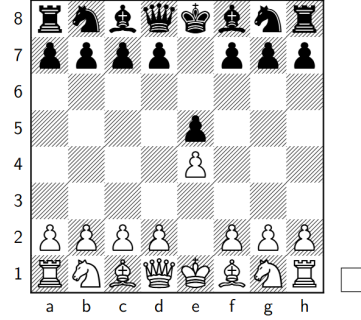
Figure 1: Chess board and notations.

1 e4



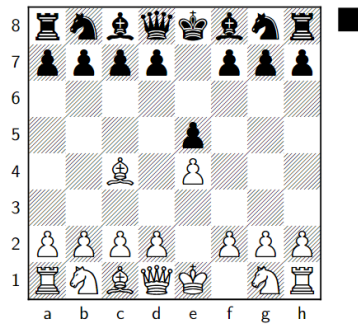
(a) Move 1

1 e4 e5



(b) Move 2

1 e4 e5 2 c4



(c) Move 3

Figure 2: Moves.